

COMP108

Algorithmic Foundations

Maximum-sum contiguous subarray

Prudence Wong

Maximum-sum contiguous subarray

Given an array $A[]$ of n numbers, find the maximum sum found in any contiguous subarray

A zero length subarray has maximum 0

Example:

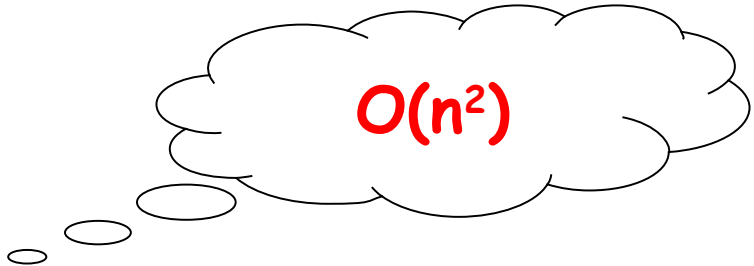
- if $A[]$ contains **-2, 5, -3, 6, -1**, then the maximum sum will be $5-3+6 = 8$
- if $A[]$ contains **10, 15, -3, -4, -2, -1, 8, 5**, then the maximum sum will be **28** (all numbers added together)

$O(n^2)$, $O(n \log n)$, $O(n)$ time algorithms

Brute-force algorithm

Brute-force algorithm

- All possible contiguous subarrays
 - $A[1..1], A[1..2], A[1..3], \dots, A[1..(n-1)], A[1..n]$
 - $A[2..2], A[2..3], \dots, A[2..(n-1)], A[2..n]$
 - ...
 - $A[(n-1)..(n-1)], A[(n-1)..n]$
 - $A[n..n]$
- How many of them in total?
- Algorithm: For each subarray, compute the sum. Find the subarray that has the maximum sum.



$O(n^2)$

Idea of brute-force algorithm

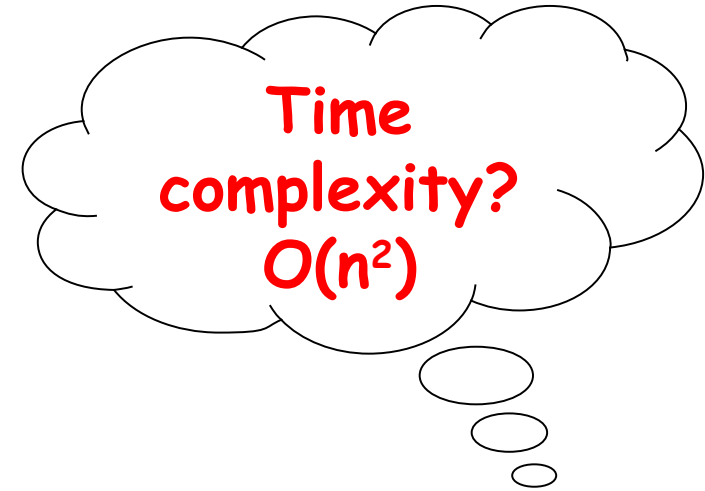
Example:	2	-6	-1	3	-1	2	-2
sum from A[1]:	2	-4	-5	-2	-3	-1	-3
sum from A[2]:		-6	-7	-4	-5	-3	-5
sum from A[3]:			-1	2	1	3	1
sum from A[4]:				3	2	4	2
sum from A[5]:					-1	1	-1
sum from A[6]:						2	0
sum from A[7]:							-2

Brute-force algorithm

- Outer loop: index variable i to indicate start of subarray, for $1 \leq i \leq n$, i.e., $A[1], A[2], \dots, A[n]$
 - for $i = 1$ to n do ...
- Inner loop: for each start index i , we need to go through $A[i..i], A[i..(i+1)], \dots, A[i..n]$
 - use an index j for $i \leq j \leq n$, i.e., consider $A[i..j]$
 - for $j = i$ to n do ...

Pseudo code

```
max = 0
for i = 1 to n do
begin
    sum = 0
    for j = i to n do
begin
    sum = sum + A[j]
    if sum > max
    then max = sum
end
end
end
```



Divide-and-conquer algorithm

Divide-and-conquer algorithm

- maximum contiguous subarray in $A[1..n]$, let $m=n/2$
 - a. max contiguous subarray in $A[1..m]$
 - b. max contiguous subarray in $A[m+1..n]$
 - c. max contiguous subarray that spans across $A[m]$
- (a) & (b) can be found **recursively**
- (c) can be found in two steps
 - consider $A[i..m]$ by fixing m but change i , i.e., $1 \leq i \leq m$
 - consider $A[(m+1)..j]$ by fixing $(m+1)$ but change j , i.e., $m+1 \leq j \leq n$
 - sum these two maximum

Idea of divide-and-conquer

Example:	10	15	-3	-4		-2	-1	8	5
max on L (recursion)	<u>10</u>	<u>15</u>							
max on R (recursion)								<u>8</u>	<u>5</u>
mid extend to L	<u>10</u>	<u>15</u>	<u>-3</u>	<u>-4</u>					
mid extend to R						<u>-2</u>	<u>-1</u>	<u>8</u>	<u>5</u>

➤ Possible candidates:

➤ 25, 13, 28 (=18+10)

➤ overall maximum **28**, i.e., take all numbers

Idea of divide-and-conquer

Example: -2 5 -1 | -5 2 -1 2

max on L (recursion) 5

max on R (recursion) 2 -1 2

mid extend to L 5 -1

mid extend to R not take any

➤ Possible candidates:

➤ 5, 3, 4 (=4+0)

➤ overall maximum **5**

Pseudo code

```
Algorithm MaxSum(A, p, q)
    if p == q then return A[p] ;
    m = ⌊(p+q)/2⌋;
    maxL = MaxSum(A, p, m) ;
    maxR = MaxSum(A, m+1, q) ;
    compute maxMidL ;
    compute maxMidR ;
    maxMid = maxMidL + maxMidR;
    return max(maxL, maxR, maxMid) ;
```

Pseudo code

Algorithm MaxSum(A, p, q)

if $p == q$ then return $A[p]$;

$m = \lfloor (p+q)/2 \rfloor$

$\text{maxL} = \text{MaxSum}(A, p, m)$;

$\text{maxR} = \text{MaxSum}(A, m+1, q)$;

compute maxMidL ;

compute maxMidR ;

$\text{maxMid} = \text{maxMidL} + \text{maxMidR}$;

return $\max(\text{maxL}, \text{maxR}, \text{maxMid})$;

$\text{sum} = 0, \text{maxMidL} = 0$
for $i = m$ down to p
begin
 $\text{sum} += A[i]$
 $\text{maxMidL} = \max(\text{maxMidL}, \text{sum})$
end

Pseudo code

Algorithm MaxSum(A, p, q)

if $p == q$ then return $A[p]$;

$m = \lfloor (p+q)/2 \rfloor$

$\text{maxL} = \text{MaxSum}(A, p, m)$;

$\text{maxR} = \text{MaxSum}(A, m+1, q)$;

compute maxMidL ;

compute maxMidR ;

$\text{maxMid} = \text{maxMidL} + \text{maxMidR}$;

return $\max(\text{maxL}, \text{maxR}, \text{maxMid})$;

$\text{sum} = 0, \text{maxMidL} = 0$
for $i = m$ down to p
begin
 $\text{sum} += A[i]$
 $\text{maxMidL} = \max(\text{maxMidL}, \text{sum})$
end

Pseudo code

Algorithm MaxSum(A, p, q)

if $p == q$ then return $A[p]$;

$m = \lfloor (p+q)/2 \rfloor$

$\text{maxL} = \text{MaxSum}(A, p, m)$;

$\text{maxR} = \text{MaxSum}(A, m+1, q)$;

compute maxMidL ;

compute maxMidR ;

```
sum = 0, maxMidR = 0
for i = m+1 to q
begin
    sum += A[i];
    maxMidR = max(maxMidR, sum)
end
```

$\text{maxMid} = \text{maxMidL} + \text{maxMidR}$;

return $\max(\text{maxL}, \text{maxR}, \text{maxMid})$;